

Project Goal

The original main goal was to use all the available digitized Census data and create an Askey family GEDCOM that is as close to perfect as possible.

The secondary goal is to be able to share this with others. All the source code is on GitHub. Anyone downloading the source code is almost required to enlist the help of an AI code assist, a very smart AI that understands math and databases and Software development. A human *can* do it. I can't do it by myself.

And I plan to offer my services if someone wants to give me their family GEDCOM. I will run it through our process and give them back a GEDCOM that I consider better.

The plan assumes the census data is truth, and we will never change the census data. We might make intermediate databases that merge and collate some, but we will not alter the actual data.

Since the digitized census data lacks most of the names, we will be merging in GEDCOM and additional death data files like BIRLS files and state/county marriage data lists to add to the database. These are only auxiliary data, and no new people will be generated from these. Only auxiliary data will be added to existing census individuals. At no time does auxiliary data override census truth. The final product will point to the census record where that person first showed up in the US system.

Processing Pipeline Steps

Here is a clean, organized list of every key Python script we've built or discussed, broken down by the phase of the pipeline it belongs to. This should be perfect for transferring over to your slides:

Phase 1: The Raw Data Foundation

Before we can verify a family tree, we must build the historical reality it will be tested against. This phase sets up the "Muscle."

DatabaseVault.py - (The Ingestion Engine): You start by downloading massive, raw CSV files from the IPUMS website containing 100 years of U.S. Census data (1850–1950). This script uses multi-threading to chew through those massive files, clean up the messy text strings, and safely lock them into pristine, read-only SQLite databases (**MasterVault_ALL.db** for the 100% nameless data, and **MasterVault_ALLs.db** for the 5% named sample). This ensures the raw historical data is never altered or corrupted by the AI.

What it does: The ingestion engine. It reads the massive, raw CSV files downloaded from the IPUMS website (covering 100 years of the U.S. Census) and securely loads them into read-only SQLite databases (**MasterVault_ALL.db** and **MasterVault_ALLs.db**). This guarantees our historical baseline is never altered.

The samples database provides names, plus it's a much smaller and easier database to view in DB viewer when making engineering decisions.

- **Process:** Uses a Python ThreadPoolExecutor to parallelize file reading. Bypasses matching logic completely to ensure safe ingestion. Trims long strings with many blanks at the end of item. Cleans standard variables via the JSON codebook while preserving raw composite IDs (Sample + Serial + PerNum).
- **Output:** Pristine, read-only SQLite vaults: **MasterVault_ALL.db** (the 100% data) and **MasterVault_ALLs.db** (the 5% samples with restored names).
 - **Notes:** the samples database is smaller and easy to view in the DB browser. Note:
 - The **BIRLS** (Beneficiary Identification Records Locator Subsystem) database is a massive historical index of deceased U.S. military veterans and their dependents who received benefits from the Department of Veterans Affairs. It contains over 19 million.
 - The Thomas Askey Family **GEDCOM** is 1,500 family members that have been gathered using Ancestry.com over the last twenty years. records useful for genealogical and historical research.

816,654,829 individual census records from 1850 to 1950.

Phase 2: The Target Anchor

IngestGedcomFile.py

What it does: Reads your legacy manual family tree (.ged file). It extracts the individuals, cleans up messy dates, and captures the all-important FAM tags (husband, wife, and child relationships). It saves these as our foundational anchors into GedcomVault.db.

This is a nice utility script used later in the process.

Phase 3: The Initial Intersection

MergeGedcomToCleanVault.py

What it does: Creates your *CleanVault_Gedcom.db* workspace.

It executes a deterministic match to pull your GEDCOM names and their inter-generational relationships into the vault, linking them to their initial historical census records and minting their permanent St. Joe's Golden Record IDs.

The Golden Records

In the Askey Genealogy Data Engine, a Golden Record represents a single, mathematically verified human being distilled from decades of messy, error-prone historical census data. They are initially minted by the Splink AI engine, which uses probabilistic string matching to resolve 19th-century typos, and are further expanded through relational graph traversal (POPLOC/MOMLOC) to identify anonymous dependents. Stored exclusively in the pristine CleanVault.db, each Golden Record acts as an immutable anchor for the family tree. It is assigned a permanent 16-character cryptographic UUID (the "St. Joe's ID") and stores a composite list of vault_pointers (e.g., 1850_1234_1|1860_5678_2). This architecture elegantly separates the resolved identity "skeleton" from the raw historical "muscle," allowing the system to instantly map a single person back to their exact line numbers in the raw MasterVault_ALL.db to generate a perfectly cited GEDCOM file.

It covers the what (a verified human), the “how” (Splink AI and graph math), and the “why” (separating the clean skeleton from the raw data to build the GEDCOM).

Phase 4: The Graph Traversal

ExpandHouseholds.py (The Living Room Sweep)

What it does: Streams the massive 816-million-row census database to look inside the specific households of your verified ancestors. It follows the POPLOC (Father) and MOMLOC (Mother) pointers to discover unnamed children, spouses, and in-laws, dragging them into your Clean Vault and minting new Golden Records for them.

Phase 5: The Nameless Snaps (Bridging the Gaps)

python/utls/DemographicGedcomSnap.py (The Couple Snap)

python/utls/DemographicGedcomSnap.py (The Couple Snap)

What it does: Finds married couples who slipped past the initial merge due to bad census handwriting. It searches the entire country for a husband and wife living together with the exact projected ages (+/- 5 years) and merges them based on that unique demographic fingerprint.

utls/DemographicChildSnap.py (The Child Snap)

What it does: Finds isolated children (like Uncle Lemuel) who were missed. It mathematically matches them to the unnamed "John/Jane" census discoveries found during the Living Room Sweep by comparing their shared parents, sex, and birth year.

Q: Missed by whom? The census tells everybody who the kids are. It's not rocket science we shouldn't do the math of a rocket scientist. We can just look up in the raw data who the children were, who the mother was, who the father was.

utls/DemographicIndividualSnap.py (The Lone Wolf Snap)

What it does: A straight-up, highly strict SQL lookup against the 5% named sample database. It finds single/isolated individuals based purely on exact Sex, First Initial, Last Name, and Birth Year (+/- 3 years).

Phase 6: The Macro-Level AI Engines

run_analyst.py (The Splink AI Matcher)

What it does: A machine-learning probabilistic engine used for massive, nationwide fuzzy-matching when strict deterministic math fails. It weighs demographics, locations, and spelling similarities to cluster messy records.

LinkFamiliesByDemographics.py (The Time Machine)

What it does: Bridges nameless families across decades without looking at names. It projects ages forward 10 years and uses the Father's Birthplace (fbpl) and Mother's Birthplace (mbpl) as a cryptographic hash to track migrations over time.

- Inputs: **CleanVault.db** (Anchored Households) and MasterVault_ALL.db.

- Process: Bridges nameless families across decades using a 5-step DuckDB query. Projects Head/Spouse ages +10 years using high-speed Range Joins (BETWEEN). Uses FBPL (Father's Birthplace) and MBPL (Mother's Birthplace) as strict cryptographic filters to mathematically prevent Cartesian explosions.
- Output: Successfully linked multi-decade family vectors saved into ***DemographicMatches.db***.

AssignFamilyIDs.py (The Universal Web)

What it does: Uses a high-speed Union-Find graph algorithm to group every single record in the entire database into a universal Family ID.

- Inputs: **CleanVault.db** and **MasterVault_ALL.db**.
- Process: Implements a high-speed Union-Find (Disjoint Set) graph algorithm in Python. Connects intra-decade households via IPUMS SERIAL & FAMUNIT, then binds them across decades using the Golden Record vault_pointers.
- Output: Maps the entire country into unique FAM_... buckets, stored in the universal_families table inside CleanVault.db.

Phase 7: Reporting & Exporting

utils/CensusVsGedcomReport.py (The Diagnostic Diff)

What it does: Instantly generates a highly readable text report that groups your family into three buckets: mathematically proven matches, legacy people missing from the census, and brand-new AI census discoveries.

python/utils/export_gedcom.py (The Grand Finale)

What it does: The most heavily optimized script in the pipeline. It reads the Clean Vault, executes lightning-fast chunked B-Tree seeks into the raw SQLite databases, and formats everything into a strict, error-free GEDCOM 5.5.1 file. It perfectly attaches "U.S. Federal Census" citations (with exact page/line numbers) to every proven fact.

•

CompareGedcoms.py (The Ancestry Diff)

What it does: A semantic diff tool that compares your original pipeline GEDCOM against the one you eventually download from Ancestry, showing exactly what records and dates Ancestry's hint engine added or modified.